实验 1 进程控制

实验目标：模拟操作系统对进程的管理。实现进程之间的切换。

1. 基本假设：

1) 单处理机，即一次只能 1 个进程处于运行态

2) 进程切换和进程调度（操作系统内核）的执行时间忽略，只考虑进程的执行指令所花的时间

3) 程序的指令分为 cpu 指令和 io 指令两种类型。在输出结果中， cpu 代表 cpu 指令，执行 cpu 指令需要 1 个时间单位; io 代表 io 指令, 需要阻塞该进程, 执行 io 指令需要 5 个时间单位(其中, 第 1 个时间单位使用 cpu（运行态），后面 4 个时间单位使用 I/O 设备（等待态））

4) 进程的状态：就绪态(输出 READY)，运行态(输出 RUN:cpu 或者 RUN:io，取决于指令的类型)，等待态(输出 WAITING)，完成态(输出 DONE)//没有新建态

5) CPU 的状态：1（忙），null(空闲)

6) I/O 设备的状态：1（忙），null（空闲）

2. 数据结构：

1) 进程控制块 PCB，包含：(1)PID; (2)进程名字；(3)进程状态；（4）进程上下文：PC（简化为：记录执行第几条指令）

2) 以队列或链表为数据结构，组织管理所有 PCB，即任何一个进程的 PCB 必然存在于以下某一个队列里：就绪队列、运行队列、等待队列、完成队列。对进程状态的切换则是通过将其 PCB 移出原队列，放入新状态对应的队列来实现。

3. 要实现的操作：

1) 新进程的创建:分配和初始化 PCB，读取指令序列到一个数组中、初始化上下文（PC=0）和进程状态（即放入就绪队列中）

2) 进程状态的切换

a) 就绪态→运行态：被调度执行，PCB 从就绪队列到运行队列

b) 运行态→完成态：最后一条指令被执行后，从运行队列到完成队列

c) 运行态→等待态：当前执行的指令是一个 io 指令，从运行队列到等待队列，设置好等待的计时器

d) 等待态→就绪态：io 指令执行所需要等待的时间片到，还有指令未执行完，从等待队列到就绪队列

e) 等待态→完成态：io 指令执行所需要等待的时间片到，没有指令需要执行，从等待队列到完成队列

4. 实验要求：

1) 任务 1. 自动生成 k 个随机的程序文件 （输入参数：程序文件个数 k, cpu 指令所占百分比 percent，指令个数范围： [min, max]）

例如:

用户输入：3（3 个程序）,80（cpu 指令占比 80%）,5，10(每个程序的指令个数范围)

程序输出：程序文件 P1.txt、P2.txt、P3.txt。每个程序文件 Pi.txt 的格式如下(每一行表示一条指令的类型):

cpu

cpu

io

cpu

io

cpu

2) 任务 2: 读入 k 个程序文件，模拟操作系统对 k 个进程的管理。实现进程之间的切换（提示：[从时刻 0 开始]在每个时刻结束之前都需要检查每一个进程的状态，并确定该状态是否要做切换）

输出格式：第 1 列 Time 是时刻(从 1 到所有进程变成完成态的时刻)，第 2–k+1 列是 k 个进程的状态

(READY/RUN:io/RUN:cpu/WAITING/DONE)，第 k+2 列是 cpu 的状态(1/0)，第 k+3 列是 IO 设备状态(1/0)

Time	PID: 0	PID: 1	PID: 2	PID: 3	PID: 4	PID: 5	CPU	IOs
1	RUN:io	READY	READY	READY	READY	READY	1	
2	WAITING	RUN:cpu	READY	READY	READY	READY	1	1
3	WAITING	RUN:cpu	READY	READY	READY	READY	1	1
4	WAITING	RUN:cpu	READY	READY	READY	READY	1	1
5	WAITING	DONE	RUN:cpu	READY	READY	READY	1	1
6*	READY	DONE	RUN:cpu	READY	READY	READY	1	
7	READY	DONE	RUN:cpu	READY	READY	READY	1	
8	READY	DONE	DONE	RUN:io	READY	READY	1	
9	READY	DONE	DONE	WAITING	RUN:cpu	READY	1	1
10	READY	DONE	DONE	WAITING	RUN:cpu	READY	1	1
11	READY	DONE	DONE	WAITING	RUN:cpu	READY	1	1
12	READY	DONE	DONE	WAITING	RUN:cpu	READY	1	1
13*	READY	DONE	DONE	READY	RUN:cpu	READY	1	
14	READY	DONE	DONE	READY	RUN:cpu	READY	1	
15	READY	DONE	DONE	READY	DONE	RUN:cpu	1	
16	READY	DONE	DONE	READY	DONE	RUN:io	1	
17	RUN:io	DONE	DONE	READY	DONE	WAITING	1	1
18	WAITING	DONE	DONE	RUN:io	DONE	WAITING	1	2
19	WAITING	DONE	DONE	WAITING	DONE	WAITING		3
20	WAITING	DONE	DONE	WAITING	DONE	WAITING		3
21*	WAITING	DONE	DONE	WAITING	DONE	RUN:io	1	2
22*	RUN:cpu	DONE	DONE	WAITING	DONE	WAITING	1	2
23*	DONE	DONE	DONE	RUN:io	DONE	WAITING	1	1
24	DONE	DONE	DONE	WAITING	DONE	WAITING		2
25	DONE	DONE	DONE	WAITING	DONE	WAITING		2
26*	DONE	DONE	DONE	WAITING	DONE	RUN:io	1	1
27	DONE	DONE	DONE	WAITING	DONE	WAITING		2
28*	DONE	DONE	DONE	RUN:cpu	DONE	WAITING	1	1
29	DONE	DONE	DONE	RUN:io	DONE	WAITING	1	1
30	DONE	DONE	DONE	WAITING	DONE	WAITING		2
31*	DONE	DONE	DONE	WAITING	DONE	RUN:cpu	1	1
32	DONE	DONE	DONE	WAITING	DONE	DONE		1
33	DONE	DONE	DONE	WAITING	DONE	DONE		1
34*	DONE	DONE	DONE	DONE	DONE	DONE		

```
Stats: Total Time 34
Stats: CPU Busy 25 (73.53%)
Stats: IO Busy  25 (73.53%)
```

实验 2(A)地址转换

实验目标：模拟操作系统对内存的分页机制的管理。实验 2 是在实验 1 的基础上增加下列基本功能：

1) [新建态：创建进程时]为进程分配内存空间、建立页表 （模拟操作系统）
2) [运行态：执行每条指令时]做一次地址转换 （模拟 MMU）
3) [完成态：进程结束时]回收进程所占用内存（模拟操作系统）

数据结构：

1) 一个全局位示图 bitmap（位数组），用于物理页帧的分配和回收。第 i 位为 0 表示第 i 个物理页帧空闲，为 1 则表示页帧被占用。Bitmap 的长度等于物理内存空间中页帧的数量
2) 一个页表指针 PTBR，指向当前运行态进程的页表基址
3) 每个进程一个页表，用于支持每个进程的地址转换。页表项一定包括：物理页帧号、有效位。关于页表项的表示，可以用结构体，也可以用位数组
4) PCB 中增加一个指向页表的指针

实现：

1. **系统初始化：**输入系统参数
   1) [页大小]页内偏移的位数 x（例如：x=12）
   2) [虚拟地址空间大小]虚拟地址的位数 a（例如：a=16）
   3) [物理地址空间大小]物理地址的位数 b（例如：b=18）
   4) [进程大小]进程的大小范围（例如：最小页数 min_size=3，最大页数 max_size=7）

5) 生成合法虚拟地址（即不会导致处理机异常的虚拟地址）的概率 p_valid（例如：p_valid=0.8）

6) 位示图中 0 的概率 p_available（例如：p_available = 0.85）

2. **系统初始化：**随机化生成位示图并输出

3. **自动生成程序的函数中加入对每个进程虚拟地址空间的初始化**

   1) 随机生成进程的大小 y 个页(min_size <= y <= max_size)

   2) 随机生成进程在虚拟空间中的有效页集合 S。具体的：设虚拟空间一共 16 个页，进程大小为 y=4 个页。则从虚拟地址空间的 16 个页中随机选取 4 个页作为有效页集合。设 S={0,1,2,15}，那么如果虚拟地址的页号是 2，则是一个合法的地址，因为 2∈S；如果页号是 3，则会出现处理机异常，因为 3∉S。

   3) 随机生成每条指令的内存引用（注意：这样的访问模式并不符合实际的局部性原理）。每个指令包括两部分：内存引用（即一个虚拟地址）和指令类型。我们只需要在实验 1 的基础上，在指令类型后面添加上一个 16 进制（或 2 进制）的**随机的**内存引用。该地址是一个有效的内存引用（即页号∈S）的概率为 p_valid。**例如，一个自动生成程序文件 Pi.txt 如下：**

```
4 0 1 2 15
cpu     31C0
cpu     2160
io      111F
cpu     00A0
```

4. **创建进程的函数中加入：**

   1) 创建页表：初始化所有页表项（物理页帧#=-1，有效位=0）

   2) 加载程序文件：读取进程的有效页集合，加载包含内存引用的指令序列

   3) 为每个进程分配内存空间（利用位示图），并更新页表

   4) 所有进程创建完毕后请输出位示图和所有进程的页表

5. **执行指令的函数中加入地址转换：**

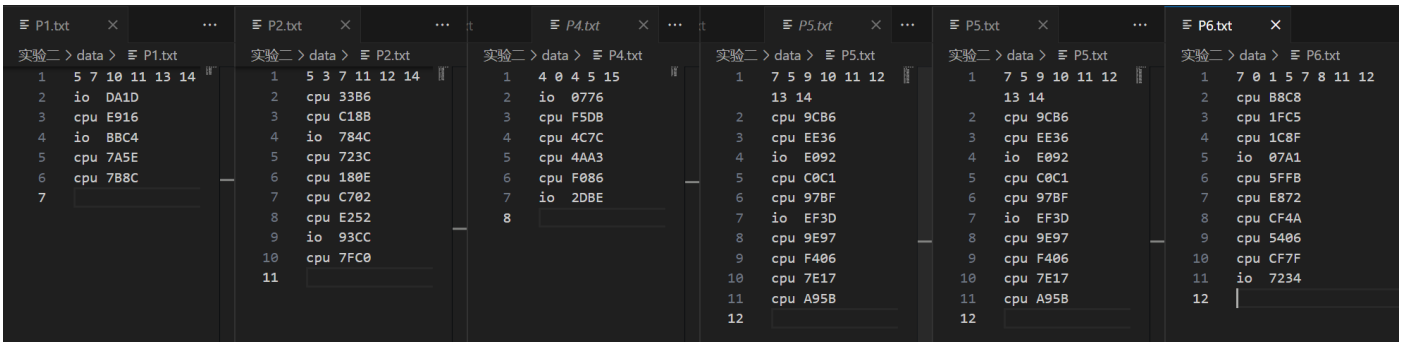
   1) 检查访问的地址是否合法，如果不合法，则终止进程的执行，将其加入完成队列

   2) 否则输出地址转换：将实验 1 中输出的 run:cpu/io 加入地址转换，如下：

run:cpu/io[16 进制的虚拟地址；16 进制的物理地址]

6. **对每个结束的进程**：回收进程占用的内存空间，更新位示图。所有程序全部结束后，输出位示图。

实验结果输出的一个样例



```
initial bitmap
00010000110100101000000000100000000111011011100001000010010010 00
after loading process
11111111111111111111111111111111111111111111111111000100001001001000

pageTable of P0
0:-     1:-     2:-     3:-     4:-     5:-     6:-     7:0     8:-     9:-     10:1    11:2    12:-    13:4    14:5    15:-
pageTable of P1
0:-     1:-     2:-     3:6     4:-     5:-     6:-     7:7     8:-     9:-     10:-    11:10   12:12   13:-    14:13   15:-
pageTable of P2
0:-     1:-     2:-     3:-     4:-     5:15    6:-     7:-     8:-     9:-     10:-    11:-    12:17   13:18   14:-    15:-
pageTable of P3
0:19    1:-     2:-     3:-     4:20    5:21    6:-     7:-     8:-     9:-     10:-    11:-    12:-    13:-    14:-    15:22
pageTable of P4
0:-     1:-     2:-     3:-     4:-     5:23    6:-     7:-     8:-     9:24    10:25   11:27   12:28   13:29   14:30   15:-
pageTable of P5
0:31    1:32    2:-     3:-     4:-     5:33    6:-     7:34    8:38    9:-     10:-    11:41   12:45   13:-    14:-    15:-

Time    PID:0                   PID:1                   PID:2                   PID:3                   PID:4                   PID:5                   CPU     IOs
1       RUN:io  [DA1D;04A1D]    READY                   READY                   READY                   READY                   READY                   1
2       WAITING                 RUN:cpu [33B6;063B6]    READY                   READY                   READY                   READY                   1       1
3       WAITING                 RUN:cpu [C18B;0C18B]    READY                   READY                   READY                   READY                   1       1
4       WAITING                 RUN:io  [784C;0784C]    READY                   READY                   READY                   READY                   1       1
5       WAITING                 WAITING                 RUN:cpu A3AC error      READY                   READY                   READY                   1       2
6*      READY                   WAITING                 DONE                    RUN:io  [0776;13776]    READY                   READY                   1       1
7       READY                   WAITING                 DONE                    WAITING                 RUN:cpu [9CB6;18CB6]    READY                   1       2
8       READY                   WAITING                 DONE                    WAITING                 RUN:cpu [EE36;1EE36]    READY                   1       2
9*      READY                   READY                   DONE                    WAITING                 RUN:io  [E092;1E092]    READY                   1       1
10      READY                   READY                   DONE                    WAITING                 WAITING                 RUN:cpu [B8C8;298C8]    1       2
11*     READY                   READY                   DONE                    READY                   WAITING                 RUN:cpu [1FC5;20FC5]    1       1
12      READY                   READY                   DONE                    READY                   WAITING                 RUN:cpu [1C8F;20C8F]    1       1
13      READY                   READY                   DONE                    READY                   WAITING                 RUN:io  [07A1;1F7A1]    1       1
14*     RUN:cpu [E916;05916]    READY                   DONE                    READY                   READY                   WAITING                 1       1
15      RUN:io  [BBC4;02BC4]    READY                   DONE                    READY                   READY                   WAITING                 1       1
16      WAITING                 RUN:cpu [723C;0723C]    DONE                    READY                   READY                   WAITING                 1       2
17      WAITING                 RUN:cpu 180E error      DONE                    READY                   READY                   WAITING                 1       2
18*     WAITING                 DONE                    DONE                    RUN:cpu [F5DB;165DB]    READY                   READY                   1       1
19      WAITING                 DONE                    DONE                    RUN:cpu [4C7C;14C7C]    READY                   READY                   1       1
20*     READY                   DONE                    DONE                    RUN:cpu [4AA3;14AA3]    READY                   READY                   1
21      READY                   DONE                    DONE                    RUN:cpu [F086;16086]    READY                   READY                   1
22      READY                   DONE                    DONE                    RUN:io  2DBE error      READY                   READY                   1
23      READY                   DONE                    DONE                    DONE                    RUN:cpu [C0C1;1C0C1]    READY                   1
24      READY                   DONE                    DONE                    DONE                    RUN:cpu [97BF;187BF]    READY                   1
25      READY                   DONE                    DONE                    DONE                    RUN:io  [EF3D;1EF3D]    READY                   1
26      READY                   DONE                    DONE                    DONE                    WAITING                 RUN:cpu [5FFB;21FFB]    1       1
27      READY                   DONE                    DONE                    DONE                    WAITING                 RUN:cpu E872 error      1       1
28      RUN:cpu [7A5E;00A5E]    DONE                    DONE                    DONE                    WAITING                 DONE                    1       1
29      RUN:cpu [7B8C;00B8C]    DONE                    DONE                    DONE                    WAITING                 DONE                    1       1
30*     DONE                    DONE                    DONE                    DONE                    RUN:cpu [9E97;18E97]    DONE                    1
31      DONE                    DONE                    DONE                    DONE                    RUN:cpu F406 error      DONE                    1
32      DONE                    DONE                    DONE                    DONE                    DONE                    DONE

Status: Total Time 32
Status: CPU Busy 31 (96.88)
Status: IO Busy 22 (68.75)

bitmap when completed
00010000110100101000000000100000000111011011100001000010010010 00
```

实验 2(B)按需调页

实验目标：模拟操作系统的虚拟存储。具体的，实现基于某种置换策略的按需调页。支持至少两种置换策略（FIFO，LRU，MIN，CLOCK，RANDOM），用户可以任意选择一种置换策略。

数据结构：用于支持页面置换策略的队列。例如，如果是 FIFO，则每个进程维护一个先进先出队列。注意：如果你选择 CLOCK 则需要维护一个全局的队列。 其他的置换策略可以为每个进程维护一个队列。

实现（在实验 2（A）的基础上增加如下）：

7. **系统初始化参数：**
   每个进程能够分配的物理页帧最大数 max_alloc （max_alloc < min_size）
   置换策略的选择：FIFO，LRU，MIN，CLOCK，RANDOM 之一
8. **创建进程：**
   和实验 2（A）不同之处在于，此时不为进程分配任何内存，但需要设置好页表：将进程的每个**有效页**对应页表项的**有效位**设置为 1，存在位设置为 0
9. **执行指令：**
   1) 如果有效位=1 且存在位=1，
      输出：hit[虚拟地址，物理地址]，命中次数+1
   2) 如果有效位=1 但存在位=0，且已分配的物理页帧数量小于 max_alloc，则分配物理页帧，按需调页：查找位示图分配空闲物理页帧，更新页表项
      输出：miss, no-replace [访问的虚拟页#，分配的物理页帧#]
      然后重新执行步骤 1）的地址转换
   3) 如果有效位=1 但存在位=0，且已分配物理页帧数量大于等于 max_alloc，则做页面置换：根

据用户所选择的置换策略，做页面置换

输出： miss，replace [访问的虚拟页#,被置换出去的页(或页帧)#]

然后重新执行步骤 1）的地址转换

10. **结束进程：**

输出各个进程的命中率

置换策略为 FIFO 输出结果如图所示

```
initial bitmap
1000110111000000110101110110010111110001011111110000100100000000

pageTable of P0
0:-     1:-     2:v     3:-     4:v     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:-    13:v    14:-    15:-
pageTable of P1
0:-     1:-     2:-     3:-     4:-     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:v    13:v    14:-    15:v
pageTable of P2
0:-     1:v     2:-     3:-     4:-     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:v    13:v    14:-    15:-

Time    PID:0                                           PID:1                                           PID:2                                           CPU     IOs
1       RUN:io  no-replace[4,01] hit[4705,01705]        READY                                           READY                                           1
2       WAITING                                         RUN:cpu no-replace[C,02] hit[C3D1,023D1]        READY                                           1       1
3       WAITING                                         RUN:cpu no-replace[F,03] hit[FB11,03B11]        READY                                           1       1
4       WAITING                                         RUN:cpu hit[C0D7,020D7]                         READY                                           1       1
5       WAITING                                         RUN:cpu replace[D,C] hit[D664,02664]            READY                                           1       1
6*      READY                                           RUN:cpu replace[C,F] hit[C06B,0306B]            READY                                           1
7       READY                                           RUN:io  replace[F,D] hit[FF18,02F18]            READY                                           1
8       READY                                           WAITING                                         RUN:cpu no-replace[1,06] hit[10AE,060AE]        1       1
9       READY                                           WAITING                                         RUN:cpu no-replace[D,0A] hit[DB06,0AB06]        1       1
10      READY                                           WAITING                                         RUN:cpu hit[10C1,060C1]                         1       1
11      READY                                           WAITING                                         RUN:cpu hit[D7D4,0A7D4]                         1       1
12*     READY                                           DONE                                            RUN:cpu hit[1880,06880]                         1
13      READY                                           DONE                                            RUN:io  replace[C,1] hit[C831,06831]            1
14      RUN:cpu hit[4E4C,01E4C]                         DONE                                            WAITING                                         1       1
15      RUN:cpu no-replace[2,02] hit[2E2D,02E2D]        DONE                                            WAITING                                         1       1
16      RUN:cpu replace[D,4] hit[DD13,01D13]            DONE                                            WAITING                                         1       1
17      RUN:cpu replace[4,2] hit[47A5,027A5]            DONE                                            WAITING                                         1       1
18*     RUN:cpu hit[4653,02653]                         DONE                                            READY                                           1
19      DONE                                            DONE                                            RUN:cpu hit[CBA6,06BA6]                         1
20      DONE                                            DONE                                            DONE

Status: Total Time 20
Status: CPU Busy 19 (95.00%)
Status: IO Busy 12 (60.00%)

Process 0 hit rate 33.33%
Process 1 hit rate 16.67%
Process 2 hit rate 57.14%

bitmap when completed
1000110111000000110101110110010111110001011111110000100100000000
```

置换策略为 LRU 输出结果如图所示

```
initial bitmap
0001010000111100001101100001000010110010001011000111111011110001

pageTable of P0
0:-     1:-     2:v     3:-     4:v     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:-    13:v    14:-    15:-
pageTable of P1
0:-     1:-     2:-     3:-     4:-     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:v    13:v    14:-    15:v
pageTable of P2
0:-     1:v     2:-     3:-     4:-     5:-     6:-     7:-     8:-     9:-     10:-    11:-    12:v    13:v    14:-    15:-

Time    PID:0                                           PID:1                                           PID:2                                           CPU     IOs
1       RUN:io  no-replace[4,00] hit[4705,00705]        READY                                           READY                                           1
2       WAITING                                         RUN:cpu no-replace[C,01] hit[C3D1,013D1]        READY                                           1       1
3       WAITING                                         RUN:cpu no-replace[F,02] hit[FB11,02B11]        READY                                           1       1
4       WAITING                                         RUN:cpu hit[C0D7,010D7]                         READY                                           1       1
5       WAITING                                         RUN:cpu replace[D,F] hit[D664,02664]            READY                                           1       1
6*      READY                                           RUN:cpu hit[C06B,0106B]                         READY                                           1
7       READY                                           RUN:io  replace[F,D] hit[FF18,02F18]            READY                                           1
8       READY                                           WAITING                                         RUN:cpu no-replace[1,04] hit[10AE,040AE]        1       1
9       READY                                           WAITING                                         RUN:cpu no-replace[D,06] hit[DB06,06B06]        1       1
10      READY                                           WAITING                                         RUN:cpu hit[10C1,040C1]                         1       1
11      READY                                           WAITING                                         RUN:cpu hit[D7D4,067D4]                         1       1
12*     READY                                           DONE                                            RUN:cpu hit[1880,04880]                         1
13      READY                                           DONE                                            RUN:io  replace[C,D] hit[C831,06831]            1
14      RUN:cpu hit[4E4C,00E4C]                         DONE                                            WAITING                                         1       1
15      RUN:cpu no-replace[2,01] hit[2E2D,01E2D]        DONE                                            WAITING                                         1       1
16      RUN:cpu replace[D,4] hit[DD13,00D13]            DONE                                            WAITING                                         1       1
17      RUN:cpu replace[4,2] hit[47A5,017A5]            DONE                                            WAITING                                         1       1
18*     RUN:cpu hit[4653,01653]                         DONE                                            READY                                           1
19      DONE                                            DONE                                            RUN:cpu hit[CBA6,06BA6]                         1
20      DONE                                            DONE                                            DONE

Status: Total Time 20
Status: CPU Busy 19 (95.00%)
Status: IO Busy 12 (60.00%)

Process 0 hit rate 33.33%
Process 1 hit rate 33.33%
Process 2 hit rate 57.14%

bitmap when completed
0001010000111100001101100001000010110010001011000111111011110001
```

实验 3 非常简单的文件系统(VSFS)

按照下述磁盘组织结构和空闲空间的管理方式，实现一个支持下面 API 操作的文件系统 VSFS：

mkdir("/a") **– 创建一个新的目录，路径名为/a**

creat("/a/b") – 创建一个新的（空的）文件，路径名为/a/b

fd=open("/a/b"); write(fd); close(fd)[1]; – 打开文件/a/b，对文件追加一个块，再关闭文件

link("/a/b", "/c/d") – 对文件/a/b 创建一个硬链接

unlink("/a/b") – 对一个文件删除一个硬链接 （如果硬链接#=0 则从磁盘上移除该文件）

注 1：为了简化，可以假设 open-write-close 为一个操作。

假设：根目录的 inode 号是 0；每个目录文件的大小不超过一个磁盘块；inode 里只有 5 个直接指针 Inode 数量为 80

1. 背景知识——磁盘的组织结构：

磁盘中的内容包括：1 个超级块（记为 S），1 个 inode 的位示图（记为 i），1 个数据块的位示图（记为 d），1 个 inode 表(记为 Inodes)，和若干数据块（记为 Data region）。我们假设，整个磁盘大小为 256KB，该文件系统的磁盘块大小设为 4KB，则可以计算出该磁盘一共有 64 个磁盘块，其中超级块占 1 个磁盘块，两个位示图分别各占 1 个磁盘块，而 inode 表占 5 个磁盘块，。那么整个磁盘的结构如下图所示。其中：

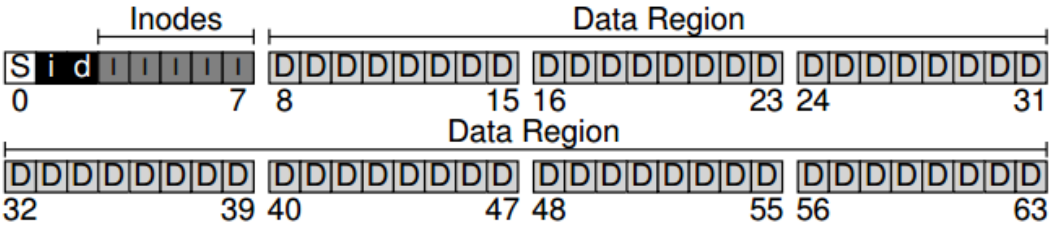


图 1 磁盘的组织结构

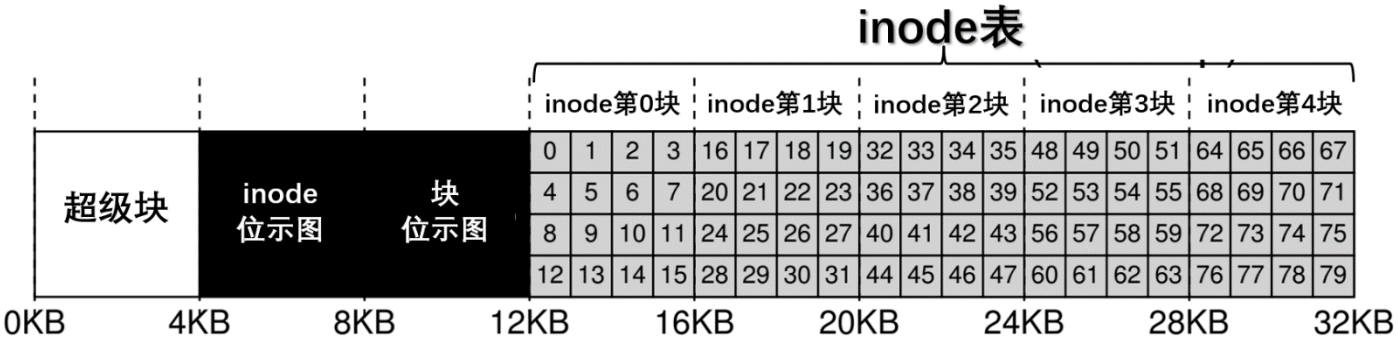


图 2 前 8 个磁盘块的信息。超级块的起始位置是 0KB，inode 位示图的起始位置是 4KB ,数据块的位示图的起始位置是 8KB，而 inode 表的起始位置是 12KB。其中，磁盘块 3 中包含了 inode[0]-inode[15]共 16 个 inodes，磁盘块 4-7 以此类推。

a) **超级块**记录了关于该文件系统的信息，包括：有多少个 inode（例如 80），多少个数据块（例如 56），inode 表的起始位置（例如，块 3）等。它还会包含一个 magic number 来表示这个文件系统的类型。

b) **inode** 是文件的元数据，它用来追踪一个文件被存放在哪些数据块，文件的大小，文件拥有者和访问权限，访问和修改的时间等。注：每个 inode 的大小是固定的。例如，我们假设每个 inode 的大小为 256 字节，那么 1 个 4KB 的磁盘块可以保存 16 个 inodes。假设我们文件系统最大支持 80 个 inodes（即最多可以有 80 个普通文件或目录文件），那么就需要 80/16=5 个磁盘块来存放整个 inode 表。在本实验中的 **inode** 有 3 个域。对于每一个 inode[j]：

- 第一个域表示文件 j 的类型（f 表示一个普通文件，d 表示一个目录）；
- 第二个域表示（普通）文件 j 的硬链接次数（如果是目录文件，则不输出）
- 第三个域表示文件存放的位置． 如果文件是空的，那么数据块地址用-1 表示（例如 a:-1）；
  如果文件有 1 个数据块，这里就有一个非负的地址(例如：a:10)；
  如果有多个数据块，就有多个地址，按顺序依次输出，(例如:a:10 11 17))。

例如，下面的 inode 所表示的文件是一个普通文件，它是空的，并且只有 1 个硬链接。

[f r:1 a:-1]

- 如果该文件有一个数据块(例如块 10)，则会变成如下：

[f r:1 a:10]

- 如果用 link("/a/b", "/c/d")系统调用又创建了一个该文件的硬链接，则会变成如下：
  [f r:2 a:10]
- 如果用 **open("/a/b"); write(fd); close(fd)** 该文件被追加写了 2 个数据块(例如块 11,17)，则会变成如下：
  [f r:2 a:10 11 17]

c) 用**两个位示图**：i-bitmap 和 d-bitmap（0-表示空闲，1-表示占用）来记录空闲的 inodes 和空闲的数据块。例如，当我们创建一个文件，我们必须给这个文件分配一个 inode。因此文件系统要扫描 i-bitmap，找到一个空闲的 inode 分配给该文件；同时把这个 i-bitmap 的相应位标记为 1。为文件分配数据块的操作类似。

d) 最后是**数据块**：数据块要么是普通文件要么是目录文件。在 VSFS 里，假设每个目录文件大小不会超过一个块（数据块里没有索引块，这是因为在这里的 inode 只包含直接指针）。如果是目录文件的数据，这个块的每一项的格式是：（文件名，文件号）。

例如，一个空的根目录文件所在的数据块看上去如下：

[(.,0) (..,0)]

如果我们加入一个普通文件“f”到根目录，被分配了文件号=1，那么存放根目录的块就变成：

[(.,0) (..,0) (f,1)]

如果一个数据块存储普通文件，则输出文件的内容。如果数据块是空的（未分配的），那么就是表示为[]。

e) **放到一起**，整个文件系统可以表示成如下形式：

```
inode bitmap 11110000
inodes       [d a:0 r:6] [f a:1 r:1] [f a:-1 r:1] [d a:2 r:2] [] ...
data bitmap  11100000
data         [(.,0) (..,0) (y,1) (z,2) (f,3)] [u] [(.,3) (..,0)] [] ...
```

如上图所示，这个文件系统有 8 个 inodes，8 个数据块。根目录包含 3 个目录项（除了.和..以外），分别是 y,z 和 f。我们可以看到 y（文件号是 1）是一个普通的文件（类型是 f），被分配了一个数据块（地址是 1）。而数据块 1 就是文件 y 的内容：为“u”。我们也看到 z（文件号是 2）是一个空的普通的文件（地址域是-1），而 f（文件号是 3）是一个目录，也是空的。你可以从 inode 位示图看到，前 4 位被标记为已分配，而数据块的位示图的前 3 位已分配。

2. 一些 API 的实现过程

注意：API 要支持多层目录的路径名和字符串形式的文件名

1. **创建文件 creat(/x/y/z)**
   a) 检查路径是否合法。例如：是否存在目录/x/y？如果不存在则报错。如果存在，检查该目录项是否已经存在？如果存在，则报错。如何合法，则继续
   b) 查找 inode 位示图，分配一个空闲的 inode 号给文件 z
   c) 初始化文件 z 的 inode：[f a:-1 r:1] //f 代表文件类型是普通文件，a 代表地址，r 代表被引用次数（因为创建一个文件时即建立了一个硬链接）
   d) 对该文件所在目录文件添加目录项：[文件名 文件号（即 inode 号）]
2. **创建目录 mkdir(/xx/uu)**
   创建目录 mkdir()的过程和创建文件类似，只是新创建的目录文件不为空。新创建的目录文件里应该有两个目录项，分别是.（目录本身）和..（父目录）。为了支持动态删除和添加目录项，目录的结构推荐采用链表。
   注：根目录在系统初始化的时候就已经创建好了，.和..对应的都是自己本身
3. **打开文件 open(/x/y/z)** //路径解析：从文件名转换成文件号
   a) 文件系统需要从文件系统的根目录开始，遍历路径名字来定位/x/y/z 的 inode 号
4. **写文件（追加一个数据块）write(fd)**
   a) 查找数据块 bitmap，分配一个块给文件
   b) 更新 inode
   c) 更新数据块(选做)

5. 关闭文件 close(fd)（选做）

a) 系统打开文件表*中引用次数-1,如果引用次数为 0，删除该项

b) 删除在进程打开文件表*中对应的表项

3. 实验目标

1) **让用户输入参数如下：[inodes 的数量，数据块的数量]**

例如：8，16。

输出初始文件系统的状态如下：

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2] [] [] [] [] [] [] []
data bitmap   10000000
data          [(.,0) (..,0)] [] [] [] [] [] [] []
```

2) **自动或手动生成对文件系统的 k 个操作序列(如下图所示)：**

```
mkdir("/a")
creat("/a/b")
mkdir("/c")
link("/a/b", "/c/d")
unlink("/a/b")
fd=open("/a/b"); write(fd); close(fd);
fd=open("/a/b"); write(fd); close(fd);
```

3) **输出每一次操作后文件系统的状态**

例如：执行操作 create("/y")后的输出：（输出操作和执行该操作后文件系统在磁盘中的状态）

```
creat("/y");

inode bitmap  11000000
inodes        [d a:0 r:3] [f a:-1 r:1] [] [] [] [] [] []
data bitmap   10000000
data          [(.,0) (..,0) (y,1)] [] [] [] [] [] [] []
```

```
fd=open("/y"); write(fd); close(fd);

inode bitmap  11000000
inodes        [d a:0 r:3] [f a:1 r:1] [] [] [] [] [] []
data bitmap   11000000
data          [(.,0) (..,0) (y,1)] [u] [] [] [] [] [] []
```

```
link("/y", "/m");

inode bitmap  11000000
inodes        [d a:0 r:4] [f a:1 r:2] [] [] [] [] [] []
data bitmap   11000000
data          [(.,0) (..,0) (y,1) (m,1)] [u] [] [] [] [] [] []
```

```
mkdir("/f");

inode bitmap   11110000
inodes         [d a:0 r:6] [f a:1 r:1] [f a:-1 r:1] [d a:2 r:2] [] ...
data bitmap    11100000
data           [(.,0) (..,0) (y,1) (z,2) (f,3)] [u] [(.,3) (..,0)] [] ...
```

注意 1：为了演示方便，在该示例中只给出了根目录下的文件操作，而没有给出带路径文件操作
注意 2：该示例中输出了目录文件的引用次数，该项可以不输出，但需要输出普通文件的硬链接次数

选做（*）

从下面两项选择一项或两项实现，选做部分不能用 AI 实现代码。

1. 将实验 3 和实验 1“进程管理”结合

1) 为实验 1 中的每一个 io 指令指定具体的内容，例如 create(“/y”)。为了方便，可以把实验 1 非运行态进程的状态输出关闭。并把 io 指令所占比例调高，例如 70%
2) 维护一个系统打开文件表，每个表项记录着该文件被引用的次数和 inode 号
3) 为每个进程维护一个进程打开文件表，可以按照文件描述符进行索引，每个表项里对应文件在系统打开文件表中的位置。通过访问系统打开文件表找到该文件的 inode
4) 执行 open(“/x/y/z”)时，通过路径解析获取该文件的 inode 号，如果这个文件是第一次打开，则在系统打开文件表新建一项，并在进程的打开文件表中分配一个文件描述符指向系统打开文件表，返回给用户；如果该文件已经存在于系统打开文件表中，则将该文件的引用次数+1，在进程的打开文件表中分配一个文件描述符指向系统打开文件表，返回给用户

2. 支持可变长字符串的文件读写并用文件来模拟磁盘块——用定长的字符串来表示一个磁盘块

```
fd=open("/a/b");                    - 打开文件/a/b
write(fd,”hello world”,len=11);     - 对文件写
close(fd);                          - 关闭文件
read(fd,len);                       - 从文件开头读 len 这么长，输出
```

通过解析字符串的格式，理解数据的结构．例如：假设系统一个 inode 占 64 个字节，第 1-4 个字节代表文件类型，第 5-8 个字节代表引用次数，第 9-28 个字节代表 5 个直接指针，其余字节为空。